Binning and Binarization: Equal Width, Equal Frequency and k-means Binning

Contents

1. Overview . . . 2
2. Why turn numbers into categories . . . 3
3. Discretization . . . 4
   3.1 What binning is good for . . . 4
4. Kinds of discretization . . . 5
5. Equal width (uniform) binning . . . 5
6. Equal frequency (quantile) binning . . . 6
7. k-means binning . . . 9
   7.1 How k-means finds the bins . . . 9
8. The three strategies on one feature . . . 10
9. KBinsDiscretizer in scikit-learn . . . 11
10. Binning the Titanic data . . . 12
    10.1 The data and the baseline . . . 13
    10.2 Binning both features . . . 13
    10.3 The result . . . 14
    10.4 Trying every strategy and number of bins . . . 15
11. Custom binning . . . 16
12. Binarization . . . 17
    12.1 Binarizer in scikit-learn . . . 18
13. Binarization on the Titanic data . . . 19
    13.1 Why a linear model gains . . . 19
14. Summary . . . 22
15. Sources . . . 23
16. Key terms . . . 23

Where this fits: Pipeline map step 5 (Engineer features). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	**5 Features**	6 Reduce
7 Split	8 Model	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Feature engineering; Feature transformation; Column transformer.
- **Used here, taught in full later:** Outliers; K-means.
- **Leads to:** Feature construction and splitting; XGBoost.
- **Compare with:** Encoding categorical data.

1. Overview

Key point: Discretization (binning) turns a numerical feature into a few ranges; binarization turns it into just 0 or 1.

A **feature** (G-772) is an input variable (one column of the data table), the **target** (G-1949) is the output we predict, and an **observation** (G-1374) is one record (one row). Earlier Notes encoded categories as numbers. This Note goes the other way: it turns numerical features into categories. The overview of feature engineering introduced the idea briefly as binning; here we cover the techniques in full.

Figure 1 shows the whole topic: one column of numbers, cut either into several bins or into just two values. There are two techniques:

- **Discretization** (G-619), also called **binning** (G-307): cut the range of a feature into several intervals and replace each value with the interval it falls in.
- **Binarization** (G-300): compare each value with one **threshold** (G-1971) and replace it with 0 or 1.

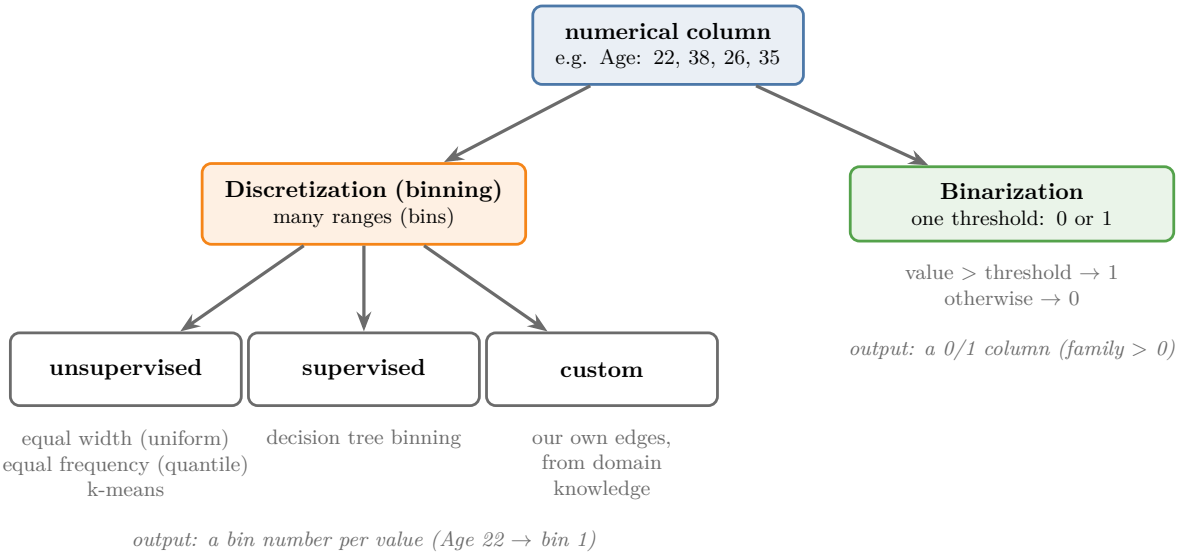


Figure 1: A numerical column can be cut into many bins (discretization) or into two values, 0 and 1 (binarization)

Discretization has several kinds (Section 4). This Note covers:

- the three unsupervised kinds: equal width (Section 5), equal frequency (Section 6) and k-means (Section 7), all provided by scikit-learn's `KBinsDiscretizer` (Section 9);
- **custom binning** (G-522) (Section 11);
- binarization, with scikit-learn's `Binarizer` (Sections 12 and 13).

2. Why turn numbers into categories

> **Key point:** Sometimes a **feature** (G-772) is easier to use as a few groups than as exact numbers; the benefit depends on the problem.

A numerical feature is usually useful as it is. Sometimes, though, the same information is better represented as categories.

Take the Google Play Store data, with a feature for each app's number of downloads. A few very famous apps have billions of downloads, while most apps have very few. The exact numbers spread over a huge range and are hard for a model to use.

Grouping the feature into categories such as "1,000+ downloads", "10,000+ downloads" and "1 million+ downloads" simplifies the problem. Download ranges are exactly how the Play Store itself shows download counts.

Whether grouping helps is problem-specific. **Binning** (G-307) is one more tool to try, not a step every project needs.

3. Discretization

> **Key point: Discretization** (G-619) cuts a feature's range into intervals and replaces each value with the interval it belongs to, just like building a histogram.

Discretization is the process of transforming a continuous variable into a discrete one by creating a set of contiguous intervals that span the range of the variable's values. The intervals are called **bins**, so discretization is also called **binning**.

Discretization works like a histogram. Take ages from 0 to 80:

1. Choose the intervals: 0 to 10, 10 to 20, 20 to 30, and so on.
2. Put each person into the interval their age falls in.
3. Each age is now replaced by its interval, or by the interval's number: 0 for "0 to 10", 1 for "10 to 20", and so on.

Counting the people in each interval gives the histogram of the feature. Discretization keeps the interval number for every **observation** (G-1374) instead of only the counts.

Figure 2 runs these steps on the 571 Titanic training ages with 10-year bins. Watch each passenger leave the waiting line and become a square in its bin's column: the stacks grow into the histogram, and every square keeps its bin number, such as age 33 in bin 3.

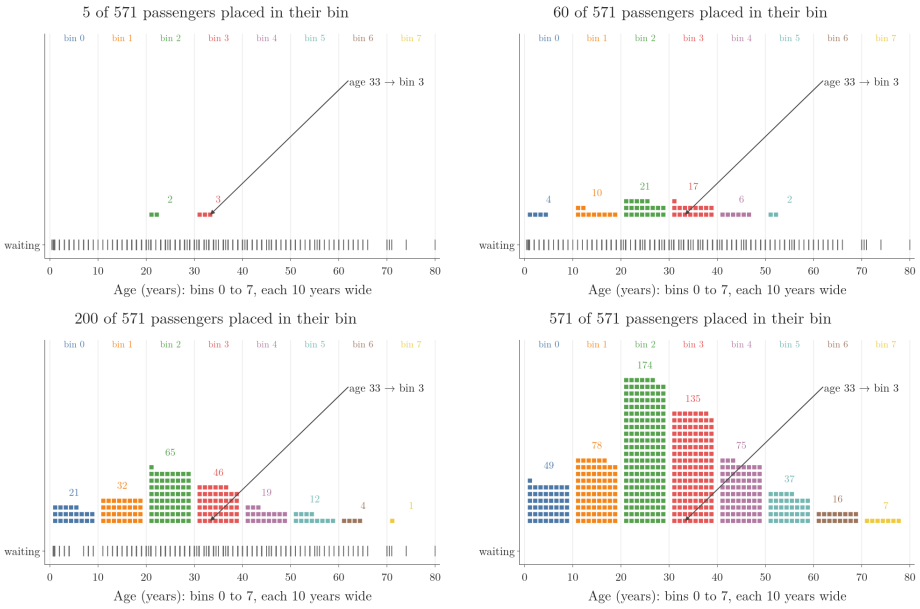


Figure 2: The 571 training ages dropped one by one into 10-year bins: the stack heights are the histogram, and each passenger keeps its bin number

3.1 What binning is good for

> **Key point: Binning** (G-307) handles outliers and can make the values spread evenly.

Binning has two benefits:

- **Handles outliers** (G-1420). A very large value lands in the last bin, together with the other large values. The large value is then treated exactly like them, so its extreme size no longer matters.
- **Improves the value spread.** Some kinds of binning put about the same number of **observations** (G-1374) in every bin. A feature whose values are bunched in one place then becomes spread evenly over its range. This helps because a model can only tell observations apart if they land in different bins: when most observations share one bin, their differences are lost (Galli 2023). Section 8 shows it on Fare, where equal width bins put 530 of the 571 passengers in the first bin.

4. Kinds of discretization

> **Key point:** Unsupervised binning looks only at the feature; supervised binning also uses the **target** (G-1949); custom binning uses edges we choose ourselves.

Discretization (G-619) comes in three kinds, shown in Figure 1:

1. **Unsupervised binning** (G-2057) uses only the values of the **feature** (G-772). Unsupervised binning has three techniques:
   - **equal width binning** (G-701), also called **uniform binning**;
   - **equal frequency binning** (G-699), also called **quantile binning**;
   - **k-means binning** (G-995).
2. **Supervised binning** (G-1918) also uses the target. Its main technique is **decision tree binning**, which needs **decision trees** (G-561) and is left for later.
3. **Custom binning** (G-522): we choose the bin edges ourselves, from knowledge of the domain.

In practice, one of the three unsupervised techniques is used most of the time. Sections 5 to 9 cover them, and Section 11 covers custom binning.

5. Equal width (uniform) binning

> **Key point:** Every bin has the same width: the range of the **feature** (G-772) divided by the number of bins.

In **equal width binning**, we first decide how many bins we want. The feature's range is then cut into that many intervals of the same width.

The bin width, step by step:

1. **In words:** subtract the smallest value from the largest, and divide by the number of bins.
2. **Formula:** with k bins,
$$\text{width} = \frac{\max - \min}{k}$$
3. **Example:** the ages of the Titanic training passengers run from 0.42 (a baby) to 80.

With 5 bins,

$$\text{width} = \frac{80 - 0.42}{5} = 15.92.$$

Starting at 0.42 and adding 15.92 each time gives the edges 0.42, 16.34, 32.25, 48.17, 64.08 and 80.

Each age then goes into its bin: an age of 23.1 falls between 16.34 and 32.25, so it lands in bin 1 (counting from 0). The left column of Figure 3 shows these five bins on the real Age feature.

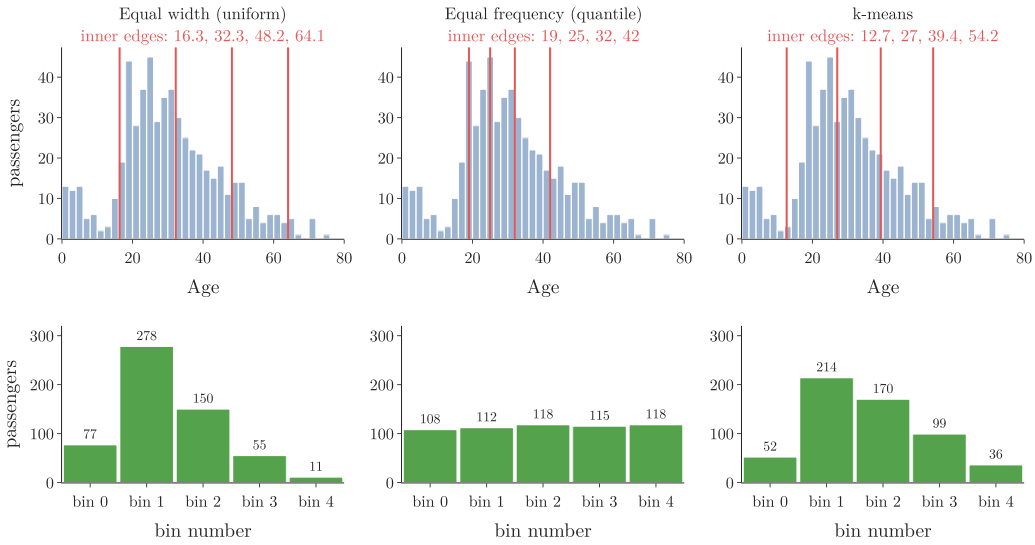


Figure 3: The three binning strategies on the same column, Age (571 training passengers, 5 bins): red lines are the bin edges (top); the number of passengers in each bin (bottom)

Equal width **binning** (G-307):

- **handles outliers:** the largest values all land in the last bin and are treated like the rest of it;
- **does not change the spread of the data:** the counts per bin follow the shape of the original histogram. In Figure 3, bin 1 holds 278 passengers and bin 4 only 11.

6. Equal frequency (quantile) binning

> **Key point:** Every bin holds about the same number of **observations** (G-1374); the edges are the feature's quantiles.

In **equal frequency binning**, we again choose the number of bins. This time each bin holds the same share of the observations: with 10 bins, each one holds 10% of them.

The plain idea. Sort the values, then draw lines that cut the sorted values into groups

of equal size. Figure 4 does so on 20 Titanic training ages (the first 20 distinct ages in the training set). Watch the red lines:

1. One line with 10 ages on each side is the median, 28.5.
2. Three lines make four groups of 5 ages.
3. Four lines make five groups of 4 ages. These four lines, at 18.8, 25.6, 30.4 and 35, are the edges of 5 equal frequency bins.



Figure 4: Quantiles as cutting lines on 20 sorted ages: the median, then the quartiles, then the four edges of 5 equal frequency bins; idea after StatQuest, "Quantiles and Percentiles, Clearly Explained!!!"

The standard term. A line that cuts sorted data into equal-sized groups is a **quantile** (G-1599). A quantile is named by the fraction of the values below it: the median is the 0.5 quantile, written $Q(0.5)$. A percentile (Section 7.2 of the understanding your data) is the same thing written as a percentage: the 0.5 quantile is the 50th percentile.

So the edges of equal frequency bins are the **feature** (G-772)'s quantiles. With 10 bins, the first edge is the 10th percentile, the second edge the 20th percentile, and so on up to the 90th.

The bin edges, step by step:

1. **In words:** with k bins, the i-th inner edge is the value below which a fraction i/k of the observations lie.
2. **Formula:**

$$e_i = Q\left(\frac{i}{k}\right), \quad i = 1, \ldots, k-1,$$

where e_i is the i-th inner edge and $Q(p)$ is the value with a fraction p of the feature's values below it.

3. **Example:** for Age with 5 bins, the inner edges are the 20th, 40th, 60th and 80th percentiles:

$$Q(0.2) = 19$$
$$Q(0.4) = 25$$
$$Q(0.6) = 32$$
$$Q(0.8) = 42$$

So about 20% of passengers are younger than 19, about 40% are younger than 25, and so on.

The middle column of Figure 3 shows the result. The bins have very different widths: 0.42 to 19 is wide, 19 to 25 is narrow. Their counts, though, are almost equal: 108, 112, 118, 115 and 118.

The counts are not exactly equal because many passengers share the same age. All passengers aged exactly 25 must go into the same bin.

Figure 5 shows where the edges come from. We sort the 571 ages from youngest to oldest and cut the sorted list into 5 piles of about 114 passengers each; the age at each cut is a quantile. Watch the flat steps: a run of passengers with the same age cannot be split, which is why the piles hold 108 to 118 passengers instead of exactly 114.

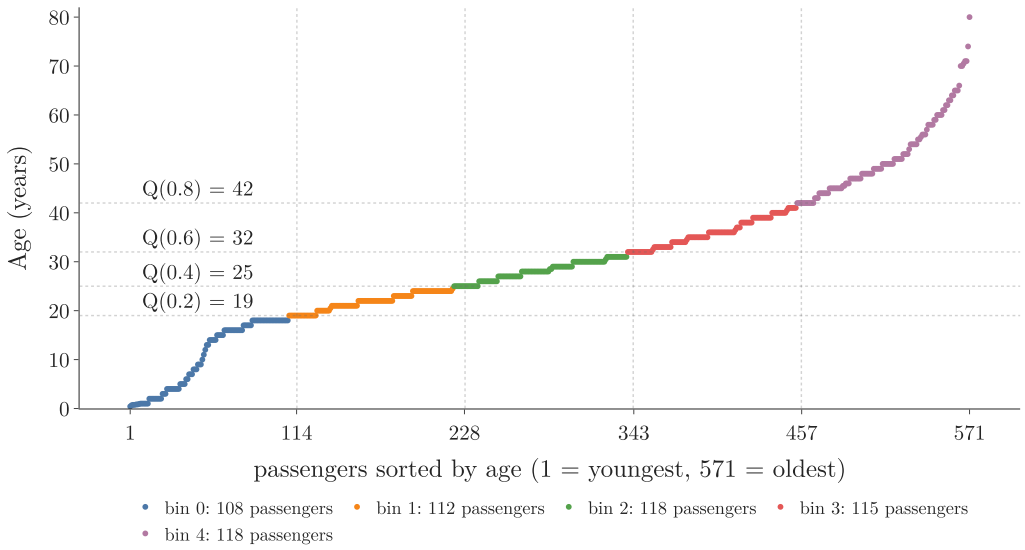


Figure 5: The 571 training ages in sorted order: cutting the list into five equal piles (vertical lines) gives the quantile edges 19, 25, 32 and 42 (horizontal lines)

Equal frequency **binning** (G-307) is used more often than equal width binning, for two reasons:

- **Equal frequency handles outliers**, like **equal width binning** (G-701).

- **Equal frequency makes the value spread uniform.** The histogram of the bin numbers is flat, as in the bottom middle of Figure 3.

Equal frequency is also scikit-learn's default strategy (scikit-learn docs, `KBinsDiscretizer`).

7. k-means binning

> **Key point:** k-means finds groups of nearby values; the bin edges are placed halfway between the groups' centres.

k-means binning uses a clustering algorithm called **k-means** (G-996; see the five steps of k-means). Clustering means finding groups of points that lie close together, called clusters. k-means finds them in data of any number of dimensions; here the data is a single **feature** (G-772), so the points lie on a line.

k-means binning (G-995) works best when the feature's values already form clusters: a group of values, then a gap with no values, then another group. On other data, the first two strategies do the job.

7.1 How k-means finds the bins

> **Key point:** Place k centres, assign each value to its nearest centre, move each centre to the mean of its values, and repeat until nothing changes.

Figure 6 runs k-means on 12 values with $k = 3$. The centres are also called **centroids** (G-367).

1. **Place k centres.** k-means places them at random. In Figure 6 they start at 14, 22 and 40.
2. **Assign each value to its nearest centre.** We compute the distance from every value to every centre, and each value joins the closest one.
3. **Move each centre to the mean of its values.** For example, the blue centre's six values are 2, 5, 8, 10, 12 and 15, with mean 8.67, so the centre moves to 8.67.
4. **Repeat steps 2 and 3** until no value changes its group.

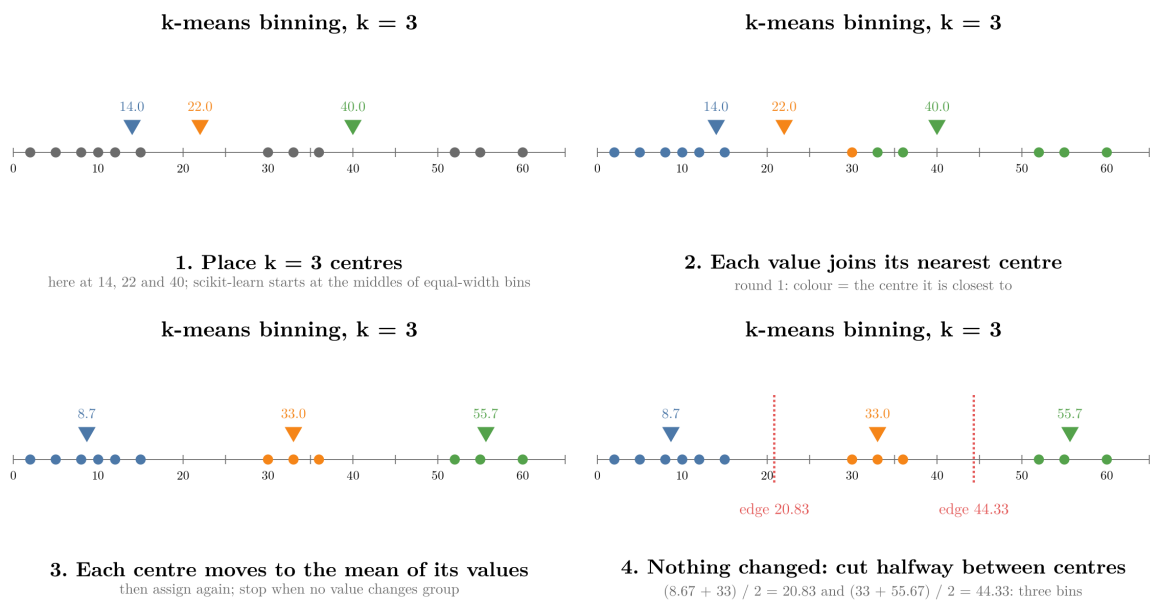


Figure 6: k-means binning on 12 values with k = 3: place centres, assign, move, repeat, then cut halfway between centres

In Figure 6, the second round moves 33 and 36 from the green group to the orange one. After the centres move again, to 8.67, 33 and 55.67, no value changes group, so the algorithm stops.

Each final group is one bin. The edge between two neighbouring bins lies halfway between their centres:

$$\frac{8.67 + 33}{2} = 20.83$$

$$\frac{33 + 55.67}{2} = 44.33$$

Extra: scikit-learn's `KBinsDiscretizer` does not start from random centres. The class places the first centres at the middles of equal-width bins, so its result is the same on every run. On the 12 values of Figure 6 the class finds the same edges, 20.83 and 44.33. A badly placed random start can leave k-means stuck in a poor grouping, which this fixed start avoids on simple data.

8. The three strategies on one feature

Key point: On a skewed feature, equal width bins leave almost every **observation** (G-1374) in the first bin, equal frequency bins share the rows out evenly, and k-means falls in between.

Figure 3 compared the strategies on Age, which is close to symmetric. Figure 7 does the same on Fare, which has a long right tail (see skewness): most fares are below 50, and a few reach 512.

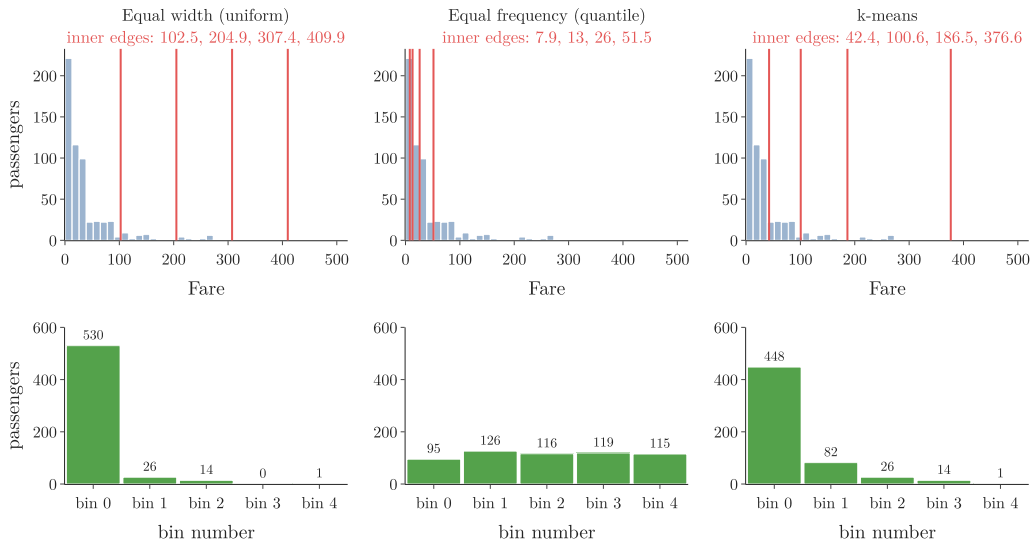


Figure 7: The three binning strategies on Fare (571 training passengers, 5 bins): bin edges (top) and passengers per bin (bottom)

Strategy	Inner edges of Fare (5 bins)	Passengers per bin
Equal width (uniform)	102.5, 204.9, 307.4, 409.9	530, 26, 14, 0, 1
Equal frequency (quantile)	7.9, 13, 26, 51.5	95, 126, 116, 119, 115
k-means	42.4, 100.6, 186.5, 376.6	448, 82, 26, 14, 1

- **Equal width:** the single fare of 512 stretches the range, so every bin is 102 wide. 530 of the 571 passengers land in the first bin, and one bin is empty.
- **Equal frequency:** the edges crowd together where the fares are dense, and every bin gets about 115 passengers.
- **k-means:** the edges follow the groups of fares. The bins are uneven, but less extreme than with equal width.

9. KBinsDiscretizer in scikit-learn

> **Key point:** `KBinsDiscretizer` needs three choices: the number of bins, the strategy and the encoding.

`KBinsDiscretizer` (G-1000) (in `sklearn.preprocessing`) performs all three unsupervised strategies. Its three main parameters are:

1. **`n_bins`:** the number of bins. The default is 5.

2. **strategy:** `"uniform"`, `"quantile"` (the default) or `"kmeans"`.
3. **encode:** how the bin of each value is written in the output.
   - `"ordinal"`: one column holding the bin number, 0, 1, 2, and so on.
   - `"onehot"` (the default): one 0/1 column per bin (as in **one-hot encoding** (G-1379)), returned as a **sparse matrix** (a table that stores only its non-zero entries).
   - `"onehot-dense"`: the same one-hot columns as an ordinary array.

The bins are in a natural order, so `"ordinal"` is the usual choice; one-hot encoding is used much less here.

Python: Binning (G-307) one column with `KBinsDiscretizer`.

```
from sklearn.preprocessing import KBinsDiscretizer

kb = KBinsDiscretizer(n_bins=5, encode="ordinal",
                      strategy="quantile")
age_bins = kb.fit_transform(X_train[["Age"]])
kb.bin_edges_[0]
# [0.42, 19, 25, 32, 42, 80]
```

bin_edges_ (G-299) holds the learned edges, one array per column, so `[0]` picks the first column's edges. `n_bins_` holds the number of bins actually made.

Extra: Details of `KBinsDiscretizer` (scikit-learn docs, `KBinsDiscretizer`).

- **Edge values:** a bin includes its left edge, so a value exactly on an edge goes to the upper bin. With edges 0, 10, 20 and 30, the value 10 lands in bin 1.
- **Values outside the training range:** the first and last edges are ignored when transforming. A test value below the smallest training value goes into bin 0, and one above the largest into the last bin.
- **Too-narrow bins are dropped:** with many tied values, two **quantiles** (G-1599) can be equal. The bin between them would be empty, so it is removed with a warning, and `n_bins_` shows fewer bins than asked for.

10. Binning the Titanic data

Key point: Binning (G-307) Age and Fare into 15 equal-frequency bins raised a decision tree's cross-validated accuracy from 63.0% to 67.5%.

In plain words, why binning can help. A **decision tree** (a model that predicts by asking a series of yes/no questions about the **features** (G-772), such as "is age above 30?"; see **decision trees** (G-561)) that sees exact ages can cut between two neighbouring ages that happen to differ in survival by chance, and learns that accident. With 15 bins, the cuts can only fall on bin edges, so the tree cannot make such fine cuts (the Extra at the end of Section 10 gives the source).

10.1 The data and the baseline

> **Key point:** Two **features** (G-772), `Age` and `Fare`; the 177 **observations** (G-1374) with a missing age are dropped, leaving 714.

We use the Titanic training file with three columns: the features `Age` and `Fare`, and the **target** (G-1949) `Survived`. Observations with a missing age are dropped, which leaves 714 passengers; 571 go to training and 143 to testing.

The baseline is a **decision tree** (G-561) on the raw numbers:

Measure	No binning
Test accuracy	62.2%
10-fold cross-validated accuracy	63.0%

The test accuracy is measured on the 143 held-out passengers. **Cross-validated accuracy** (G-509) is the average accuracy over several different splits of the data into training and test parts (10 here), so it does not depend on one lucky split (see cross-validation with a pipeline).

Python: Data, split and baseline.

```
df = pd.read_csv("data/titanic_train.csv",
                 usecols=["Age", "Fare", "Survived"])
df = df.dropna()
X, y = df[["Age", "Fare"]], df["Survived"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
clf = DecisionTreeClassifier(random_state=0)
clf.fit(X_train, y_train)
accuracy_score(y_test, clf.predict(X_test))  # 0.622
```

10.2 Binning both features

> **Key point:** One `KBinsDiscretizer` per **feature** (G-772) inside a `ColumnTransformer`, so each feature can get its own settings.

We make two `KBinsDiscretizer` objects, one for `Age` and one for `Fare`, and combine them with a `ColumnTransformer` (which applies a different transformer to each chosen column; see the easy way). Here both use 15 equal-frequency bins, but with two objects each feature could get its own number of bins or strategy.

Python: Binning (G-307) both columns.

```
kbin_age = KBinsDiscretizer(n_bins=15,
    encode="ordinal", strategy="quantile")
kbin_fare = KBinsDiscretizer(n_bins=15,
    encode="ordinal", strategy="quantile")
```

```
trf = ColumnTransformer([
    ("age", kbin_age, ["Age"]),
    ("fare", kbin_fare, ["Fare"])])
X_train_trf = trf.fit_transform(X_train)
X_test_trf = trf.transform(X_test)
trf.named_transformers_["age"].bin_edges_
```

`named_transformers_` gives back each fitted transformer by its name, so we can read its edges.

The 16 learned edges of Age are 0.42, 6, 16, 19, 21, 23, 25, 28, 30, 32, 35, 38, 42, 47, 54 and 80. Each age is replaced by its bin number:

Age	Bin number	Bin range	Fare	Bin number	Bin range
35	10	35 to 38	53.10	12	51.48 to 76.29
19	3	19 to 21	7.85	2	7.78 to 7.90
17	2	16 to 19	10.50	5	10.50 to 13.00

Python: The range of each bin can be shown with pandas' `pd.cut`, which puts values into given intervals.

```
edges = trf.named_transformers_["age"].bin_edges_[0]
pd.cut(X_train["Age"], bins=edges, right=False)
```

`right=False` makes each interval include its left edge, as `KBinsDiscretizer` does. Without it, `pd.cut` includes the right edge instead, and a value sitting exactly on an edge, such as 35, is labelled one bin lower than its bin number.

10.3 The result

Key point: With **binning** (G-307) inside a pipeline, cross-validation shows a gain of about 4.5 points.

Measure	No binning	15 quantile bins
Test accuracy	62.2%	63.6%
10-fold cross-validated accuracy	63.0%	67.5%

For cross-validation, the binning goes inside a pipeline with the model (see cross-validation with a pipeline). Each fold then learns its bin edges from its own training part.

Python: Cross-validating the binning and the model together.

```
pipe = make_pipeline(trf,
    DecisionTreeClassifier(random_state=0))
cross_val_score(pipe, X, y, cv=10).mean()  # 0.675
```

Extra: A common slip is to bin the data but pass the original `X` to `cross_val_score`. The score then belongs to the model without binning (here 63.0%), and binning seems to change nothing. Passing the pipeline, as above, scores the binned model.

10.4 Trying every strategy and number of bins

Key point: Which strategy and how many bins work best can only be found by trying; here several settings reach 67% to 69%.

To compare settings quickly, we wrap the steps in one function, `discretize(bins, strategy)`. The function:

1. bins both **features** (G-772);
2. prints the **cross-validated accuracy** (G-509);
3. plots each feature's histogram before and after.

Bins	Equal width (uniform)	Equal frequency (quantile)	k-means
5	63.2%	67.7%	67.8%
10	68.6%	67.5%	66.5%
15	65.3%	67.5%	66.3%

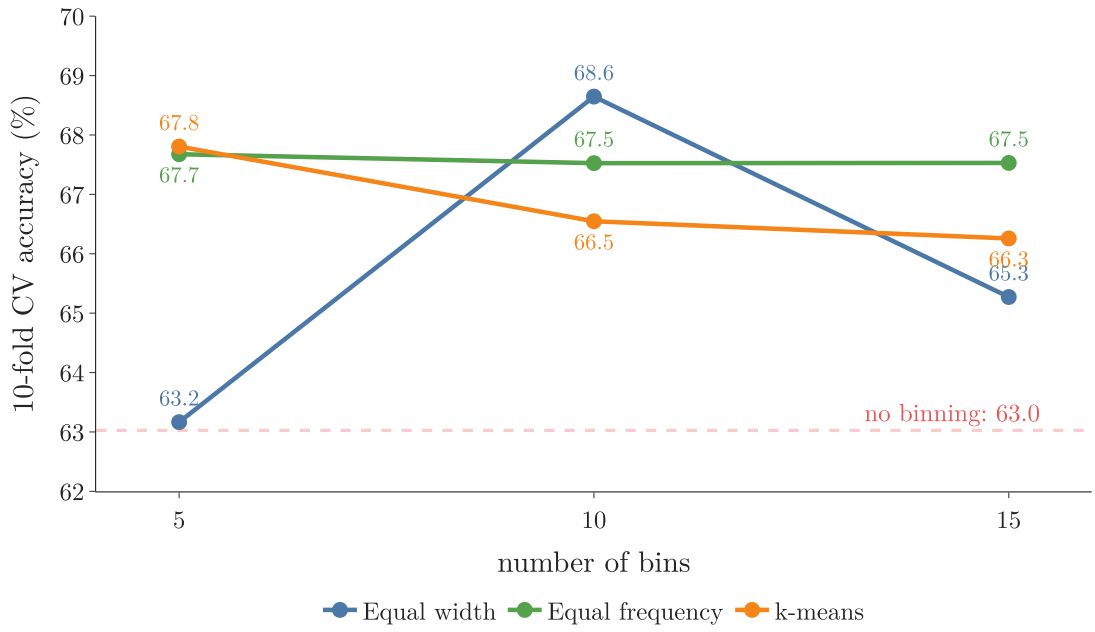


Figure 8: Cross-validated accuracy of the binned decision tree for each strategy and number of bins; the dashed line is the tree without binning

Figure 8 draws the table. Watch the green line stay flat while the blue one jumps: equal

frequency gives about the same result whatever the number of bins, and its histograms after **binning** (G-307) are flat, as expected. Equal width depends strongly on the number of bins: its histograms keep the shape of the original feature. The table is the only reliable guide.

Python: Part of the comparison function (the Notebook has the plots too).

```
def discretize(bins, strategy):
    kb = lambda: KBinsDiscretizer(n_bins=bins,
        encode="ordinal", strategy=strategy)
    trf = ColumnTransformer([("age", kb(), ["Age"]),
                             ("fare", kb(), ["Fare"])])
    pipe = make_pipeline(trf,
        DecisionTreeClassifier(random_state=0))
    print(cross_val_score(pipe, X, y, cv=10).mean())

discretize(5, "kmeans")    # 0.678
```

Extra: A decision tree is an odd model for showing binning, since a tree already cuts each feature into ranges. Binning still changes where those cuts can go, which can stop the tree from making very fine cuts that only fit the training data. Linear models can gain more: binning lets them give each range its own weight. scikit-learn's own example puts it this way: after binning, a linear model becomes much more flexible, while a decision tree becomes much less flexible (scikit-learn example, "Using **KBinsDiscretizer** (G-1000) to discretize continuous features").

11. Custom binning

Key point: In **custom binning** (G-522) we choose the bin edges ourselves from domain knowledge; scikit-learn has no class for it, so we use pandas.

In **custom binning**, also called **domain-based binning**, we set the edges ourselves, based on our knowledge of the problem. For ages, for example:

Age	Group	Reason
0 to 17	child	not yet an adult
18 to 59	adult	working age
60 and over	senior	retirement age

None of the three strategies would find these edges: they come from knowing what the ages mean. `KBinsDiscretizer` cannot take edges we choose, so we do it with pandas.

Python: Custom bins with `pd.cut`.

```
import numpy as np

age_group = pd.cut(df["Age"],
```

```
    bins=[0, 18, 60, np.inf], right=False,
    labels=["child", "adult", "senior"])
```

`bins` lists the edges; `np.inf` (infinity) makes the last bin open-ended. `labels` names the bins.

On the Titanic passengers these groups matter (Figure 9): 54% of the 113 children survived, against 39% of the 575 adults and 27% of the 26 seniors. Watch the bars fall step by step from child to senior.

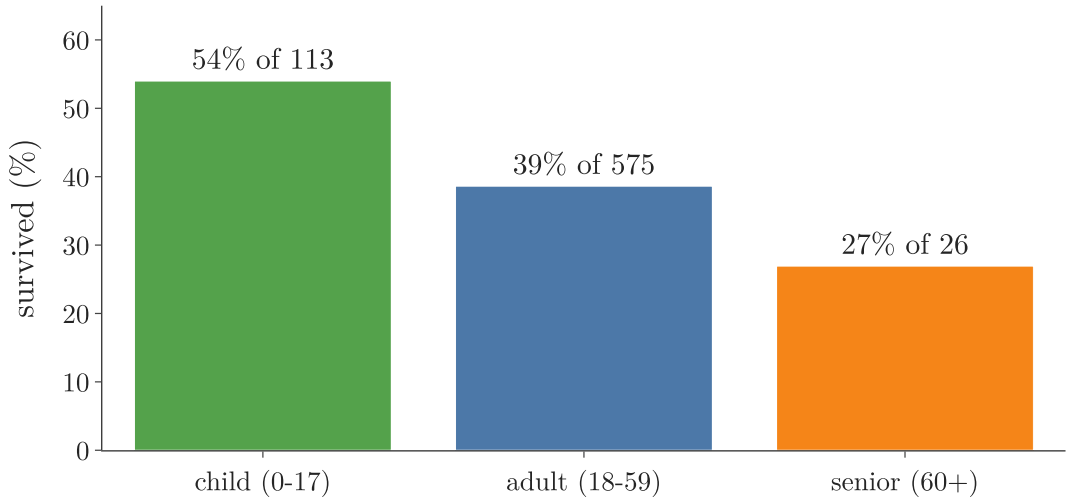


Figure 9: Survival of the three custom age groups among the 714 passengers with a recorded age

Extra: To use custom bins inside a scikit-learn pipeline, wrap the **binning** (G-307) in a `FunctionTransformer` (a transformer that applies any function to the data; see applying the log with FunctionTransformer), for example `FunctionTransformer(lambda x: np.digitize(x, [18, 60]))`. NumPy's `np.digitize` returns each value's bin number for the edges we give.

12. Binarization

Key point: Binarization (G-300) compares every value with one threshold: above it becomes 1, otherwise 0.

Binarization turns a continuous value into a binary one, 0 or 1. Binarization is a special case of **discretization** (G-619) with only two bins and one edge, the **threshold** (G-1971).

Take a **feature** (G-772) of annual income. Suppose income above 600,000 rupees is taxable:

- income above 600,000 rupees becomes 1 (taxable);

- income of 600,000 rupees or less becomes 0 (not taxable).

The income feature has become a taxable or not taxable feature.

Binarization is needed in a few specific cases. A well-known one is image processing.

A greyscale image stores each pixel as a number from 0 (black) to 255 (white). With a threshold of 127.5, every pixel at or below it becomes 0 (black) and every pixel above it becomes 1 (white). Figure 10 shows the result: a black-and-white image.

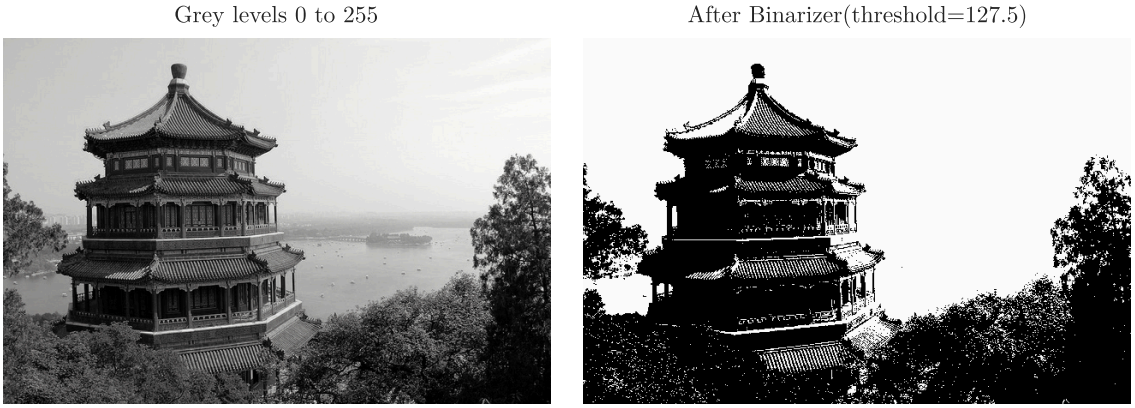


Figure 10: A greyscale image binarized with threshold 127.5: every pixel becomes black or white

12.1 Binarizer in scikit-learn

> **Key point:** `Binarizer` has two parameters: `threshold` and `copy`.

`Binarizer` (G-301) (in `sklearn.preprocessing`) performs **binarization** (G-300). Its two parameters are:

- **`threshold`** (default 0): values strictly above it become 1; values equal to it or below become 0.
- **`copy`** (default `True`): when `True`, the result is a new array and the input is left alone. When `False`, the transformer may change the input array itself.

> **Python:** Binarizing pixel values.
```
from sklearn.preprocessing import Binarizer

Binarizer(threshold=127.5).fit_transform(
    [[0, 100, 127.5, 128, 255]])
# [[0, 0, 0, 1, 1]]
```
127.5 itself becomes 0: only values strictly above the **threshold** (G-1971) become 1.

Extra: `copy=False` saves memory but is risky. With pandas 3 and scikit-learn 1.9, `ColumnTransformer([("bin", Binarizer(copy=False), ["family"])])` overwrote the `family` column of the input DataFrame itself with 0s and 1s. Running the same code twice then binarizes data that is already binarized, and the original family sizes are lost. We keep the default, `copy=True`.

13. Binarization on the Titanic data

Key point: Turning family size into "alone (0) or with family (1)" helps logistic regression, because survival rises and then falls with family size, a shape a straight-line model cannot follow.

We use `Age`, `Fare`, `SibSp` and `Parch`, and again drop the **observations** (G-1374) with a missing age (714 remain). As in the family size example, we add `SibSp` (siblings and spouses on board) and `Parch` (parents and children on board) into one **feature** (G-772), `family`, and drop the two originals.

Age	Fare	family
31.0	20.53	2
26.0	14.45	1
33.0	7.78	0

The question we want the feature to answer is: was this passenger travelling alone? A family of 0 means alone; anything above 0 means with family. Answering a yes/no question with one cut-off is binarization with **threshold** (G-1971) 0.

Python: Binarizing only the `family` feature.

```
trf = ColumnTransformer(
    [("bin", Binarizer(threshold=0), ["family"])],
    remainder="passthrough")
X_train_trf = trf.fit_transform(X_train)
X_test_trf = trf.transform(X_test)
```

`remainder="passthrough"` keeps `Age` and `Fare` unchanged. The `family` column comes first in the output, because transformed columns are placed before the passed-through ones.

After the transform, the passengers with families of 2 and 1 have `family` = 1, and the passenger travelling alone has 0.

13.1 Why a linear model gains

Key point: Survival is low for passengers alone, high for small families and low again for big ones; the 0/1 **feature** (G-772) captures the main jump.

Think of a light switch versus a dimmer. Survival behaves more like a switch: 32% of passengers travelling alone survived, against 52% of those with family. Logistic regression (a linear model that predicts a probability from a weighted sum of the features) can only draw one steady trend through the raw family size, like a dimmer turned one way. Passengers with 4 or more relatives on board survived only about 20% of the time, which drags that trend flat. The binarized feature hands the model the switch directly.

Figure 11 shows the survival rate for each family size. Watch the jump between family 0 (grey) and family 1, and the drop again from 4 relatives on; the two dashed lines are what the binarized feature keeps: 32% alone and 52% with family.

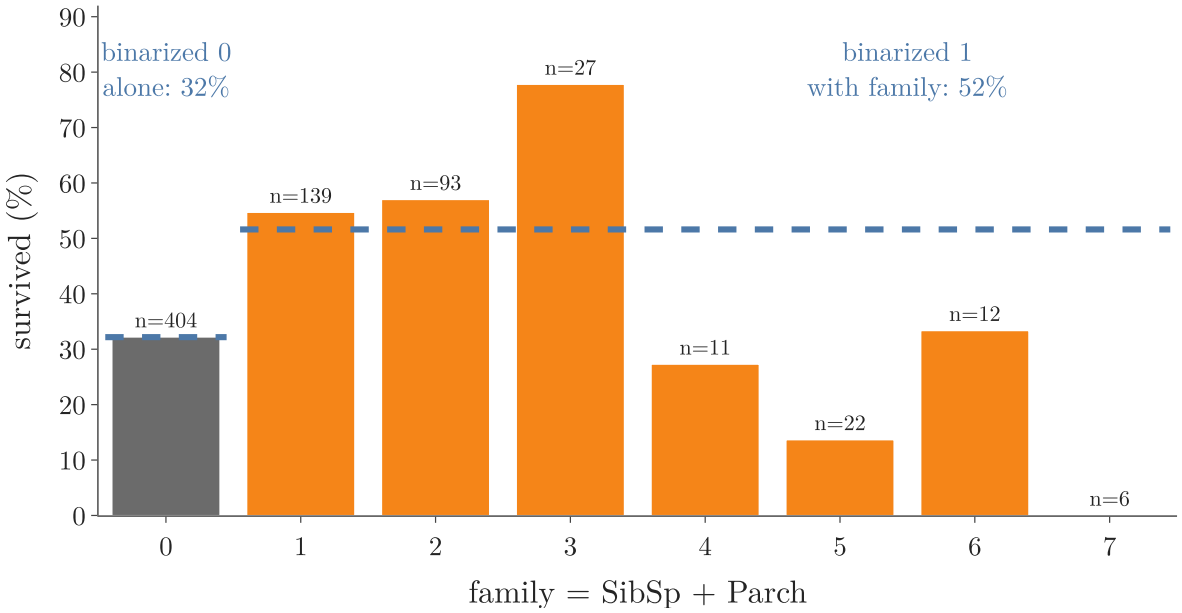


Figure 11: Survival by family size among the 714 passengers with an age; binarizing at 0 keeps the jump from alone (0) to with family (1)

We give logistic regression the `family` feature alone, first as a number, then binarized, and cross-validate each inside a pipeline:

Logistic regression on `family`	Accuracy (10-fold, cross-validated)	ROC AUC
family as a number	58.4%	0.585
family binarized	60.1%	0.599

Binarizing raises both scores. ROC AUC (area under the ROC curve), a ranking score, measures how well the model ranks survivors above non-survivors, with 0.5 meaning no better than guessing. The scikit-learn example on binning makes the same point: cutting

a feature into ranges makes a linear model more flexible (scikit-learn example, "Using **KBinsDiscretizer** (G-1000) to discretize continuous features").

> **Extra:** The numbers are averaged over 10 different shuffles of the 10-fold split, so they do not depend on one lucky split.

> **Extra:** A **decision tree** (G-561) does not gain. On `Age`, `Fare` and `family`, a decision tree scored 65.4% cross-validated with `family` as a number and 62.6% with it binarized (test set: 60.8% and 61.5%). The tree could already ask "is family above 0?" on the original feature, and binarizing removes the family size, which the tree was also using. The same scikit-learn example notes that **binning** (G-307) makes a tree *less* flexible.

So **binarization** (G-300) pays off when the model cannot draw the cut-off itself, as with linear models. Whether it helps must still be tested on each problem, as with every transformation.

14. Summary

Technique	Bins	How the edges are chosen	Spread after	Best when
Equal width (uniform)	k	range divided into k equal widths	unchanged	values spread fairly evenly
Equal frequency (quantile)	k	quantiles: same number of rows per bin	uniform	most cases; the default
k-means	k	halfway between k-means centres	uneven	values form clusters
Custom	any	chosen by us from domain knowledge	depends	the meaning of the ranges is known
Binarization	2	one threshold	0 or 1	a yes/no question about the value

- Discretization (binning) replaces each value with the interval it falls in; **binarization** (G-300) replaces it with 0 or 1. Both help when a few groups describe a feature better than exact numbers, such as download ranges on the Play Store.
- Binning **handles outliers** (G-1420), because a very large value lands in the last bin and is treated like the other large values; equal frequency binning also makes the value spread uniform, because every bin holds about the same number of rows, so no single bin swallows most rows and hides their differences.
- `KBinsDiscretizer(n_bins, encode, strategy)` does equal width, equal frequency and **k-means binning** (G-995); `bin_edges_` shows the learned edges, so we can check which range each bin number stands for.
- Custom bins need `pd.cut` (or `np.digitize`), since `KBinsDiscretizer` does not take our own edges. Custom edges matter when the ranges have a meaning, such as child, adult and senior (54%, 39% and 27% survived).
- `Binarizer(threshold)` makes values above the **threshold** (G-1971) 1 and the rest 0, so one cut-off answers a yes/no question such as "taxable?" or "travelling alone?".
- On the Titanic data, 15 equal-frequency bins raised a **decision tree** (G-561) from 63.0% to 67.5% (cross-validated), because the tree can no longer make very fine cuts that only fit the training data; binarizing family size raised logistic regression from 58.4% to 60.1%, because a linear model cannot follow survival rising and then falling with family size, while a decision tree, which can make that cut itself, did not gain.
- Cross-validate a transformation inside a pipeline, so each fold learns its bin edges from its own training part, and try several settings: no strategy is always best.
- So turning numbers into a few ranges, or into 0 and 1, is one more transformation to try: it pays off when groups carry the meaning better than exact values, and only testing shows whether it does.

15. Sources

Built from

- CampusX, "Binning and Binarization | Discretization | Quantile Binning | KMeans Binning", YouTube, https://www.youtube.com/watch?v=kKWsJGKcMvo
- StatQuest with Josh Starmer, "Quantiles and Percentiles, Clearly Explained!!!", YouTube, https://www.youtube.com/watch?v=IFKQLDmRK0Y

Other references

- Galli, S. (2023). "Master Data Binning in Python using Pandas". Train in Data blog, 23 February 2023. https://blog.trainindata.com/master-data-binning-in-python-using-pandas/ (equal width binning can put most observations in the same interval and lose information; equal frequency binning improves the value spread).
- scikit-learn documentation. `sklearn.preprocessing.KBinsDiscretizer`; example "Using KBinsDiscretizer to discretize continuous features". scikit-learn.org.

16. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Discretization (G-619)	Turning a continuous column into a discrete one by cutting its range into intervals (bins), such as ages 0–18, 18–60 and 60+; it turns endless values into a few groups that a model or a table can handle.
Binarization (G-300)	Turning a continuous column into 0 or 1 by comparing it with one threshold.
Threshold (binarization) (G-1971)	The value that separates 0 from 1 in binarization.
Custom binning (G-522)	Binning with edges we choose from domain knowledge; also called domain-based binning.
Unsupervised binning (G-2057)	Binning (grouping a column's values into ranges) that sets the bin edges from the column's own values only, without using the target.
Equal width binning (G-701)	Binning into bins of the same width, $(\max - \min)/k$; also called uniform binning.
Equal frequency binning (G-699)	Binning into bins holding the same number of rows, with the quantiles as edges; also called quantile binning.
k-means binning (G-995)	Binning whose edges lie halfway between the centres of the groups found by k-means.

Term	Meaning
Supervised binning (G-1918)	Grouping a column's values into ranges (binning) with edges chosen using the target too, such as decision tree binning.
KBinsDiscretizer (G-1000)	scikit-learn's class for equal width, equal frequency and k-means binning.
bin_edges_ (G-299)	The fitted `KBinsDiscretizer` attribute holding the learned edges.
Cross-validated accuracy (G-509)	Accuracy averaged over several train-test splits of the data, an estimate of performance on new data.
Binarizer (G-301)	scikit-learn's class for binarization, with parameters `threshold` and `copy`.
`strategy` (KBinsDiscretizer) (G-149)	The `KBinsDiscretizer` parameter that chooses how the bin edges are set: uniform, quantile or kmeans binning.
Bin edge (G-297)	A boundary between two neighbouring bins.
encode (G-680)	The `KBinsDiscretizer` parameter choosing ordinal (bin numbers) or one-hot output.
n_bins (G-1288)	The `KBinsDiscretizer` parameter for the number of bins.
pd.cut (G-1471)	The pandas function that puts values into intervals we give it.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Observation (G-1374)	One record of a dataset: one row of the data table.
Binning (G-307)	Grouping a numerical column into ranges that act as categories.
Outlier (G-1420)	A value far from the rest of the data.
Decision tree (G-561)	A model that predicts by asking a chain of questions about the input columns: nested if-else conditions.
Quantiles (G-1599)	Values that cut sorted data into equal-sized groups; a fraction p of the values lie below the quantile $Q(p)$.
k-means (G-996)	A clustering algorithm that repeatedly assigns points to the nearest centre and moves each centre to the mean of its points.
Centroid (G-367)	The centre point of one cluster in k-means, the mean of the points in it; every point is assigned to its nearest centroid.

Term	Meaning
One-hot encoding (G-1379)	Replacing a nominal column by one 0/1 column per category, with a single 1 in each row; also used to represent words.
Bin (G-298)	One of the equal ranges a histogram splits the data into.